



ParityDB-DBLP

An explainable SQL-SPARQL comparison over bibliographic knowledge graphs

Luca Tam - 2045168

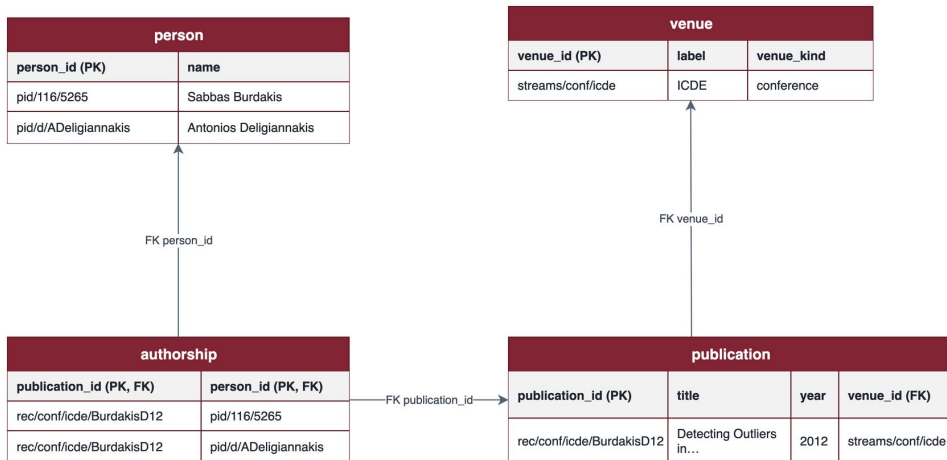
Edoardo Simonetti Spallotta - 2077561

Data Management 2025/26

Sapienza University of Rome



Relational Databases and SQL

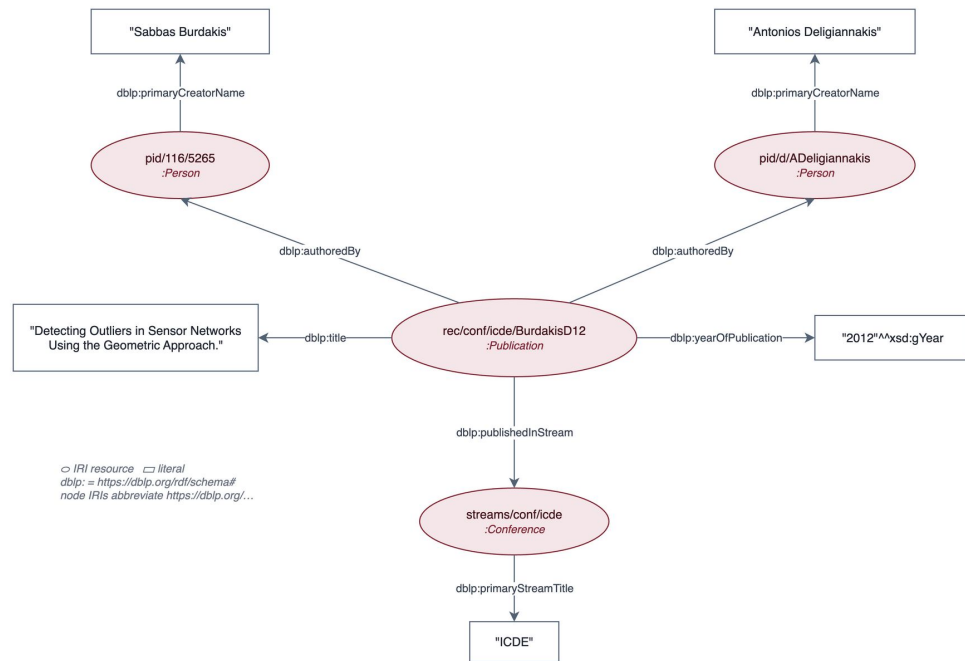


- Dominant backend since '80s
- Fixed schema, normalization, declarative SQL, mature optimizers
- Relationships exist via FOREIGN KEYS
- “Join Pains” = cost grows with table size



RDF Databases and SPARQL

- RDF databases = set of triplets (subject, predicate, object)
- **IRIs** as identifiers; literals as end nodes; no fixed schema
- **SPARQL**: graph patterns graph pattern matching, property paths, aggregation
- **Apache Jena Fuseki**: SPARQL server over an RDF dataset



Same Question, Two Languages

“Who wrote Detecting Outliers..., where and when was it published?”

SQL - PostgreSQL

```
SELECT pe.name, v.label, p.year
FROM publication p
JOIN authorship a ON a.publication_id = p.publication_id
JOIN person pe ON pe.person_id = a.person_id
JOIN venue v ON v.venue_id = p.venue_id
WHERE p.publication_id = 'https://dblp.org/rec/conf/icde/BurdakisD12';
```

3 JOINs through a bridge table

SPARQL - Fuseki

```
SELECT ?name ?venue ?year
WHERE {
  <https://dblp.org/rec/conf/icde/BurdakisD12>
    dblp:authoredBy      ?p ;
    dblp:publishedInStream ?v ;
    dblp:yearOfPublication ?year .
  ?p dblp:primaryCreatorName ?name .
  ?v dblp:primaryStreamTitle ?venue . }
```

Graph pattern mirroring the graph

Same answer: Burdakis, Deligiannakis - ICDE - 2012



Problem Statement

“Who wrote it, where, when?”

PostgreSQL
SQL

?

Fuseki
SPARQL

Backend choice lacks comparable evidence.

- Published SQL/SPARQL comparisons rarely guarantee that both systems store the same facts under the same semantics
- Backend selection guidance is typically structural and is not accompanied by measurements
- Our course itself notes the caveat: graph DB performance “depends on the kind of queries”, without specifying which





Which backend should answer a query?

Our contribution:

- **Semantic parity:** One deterministic extract feeds both stores.
- **Controlled evidence:** Wrong or unequal answers never become timing evidence.
- **A falsifiable routing policy:** Structural rules were frozen before measurement and scored by latency regret.

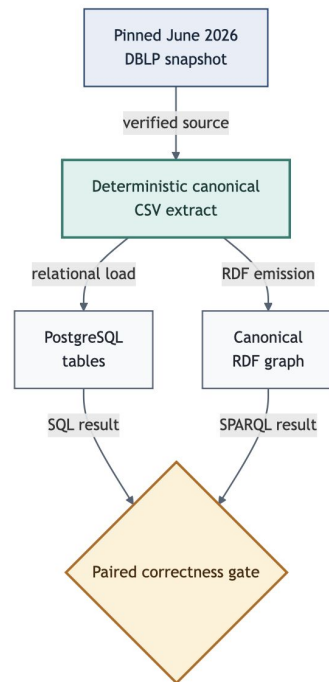


One canonical test, two representations

- 2005-2024 · 5 venues
- 42,051 publications · 66,390 persons · 173,986 authorships
- 475,002 RDF triples

All selection and exclusion policy lives in the graph in the picture.

Pinned DBLP snapshot gets filtered once into six canonical CSV files. Those exact facts are then loaded into PostgreSQL and re-serialized as RDF for Fuseki.

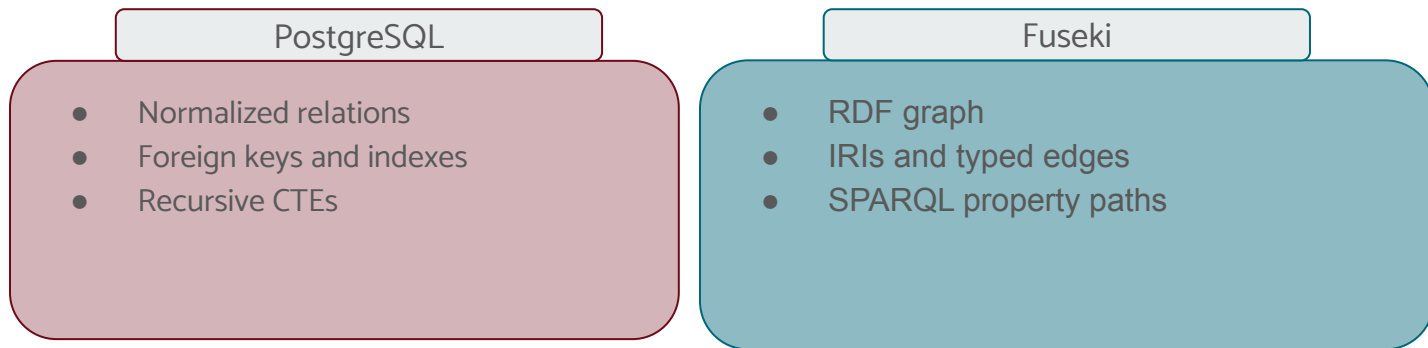




Same facts, model-specific strengths

The representations are not forced to look syntactically identical, each backend uses its idiomatic mechanisms, the conceptual fact set and meaning of every query is shared.

We also add the the same small research-area hierarchy to both stores for the hierarchy-aware analysis.



Five analytical query families

We establish two macro groups of queries:

- Q1-Q2 are baselines: trends, productivity, co-author reach
- Q3-Q5 are the richer, original analyses we introduced: distance, indirect candidates, cross-area inference.

Query	What	Goal
Q1	Publication Trends	Grouping & Aggregation
Q2	Productive, collaborative authors	Aggregation + self-join
Q3	Bounded collaboration distance	Recursive traversal
Q4	Indirect collaborator discovery	Exact two-hop pattern
Q5	Cross-area authors	Hierarchy closure

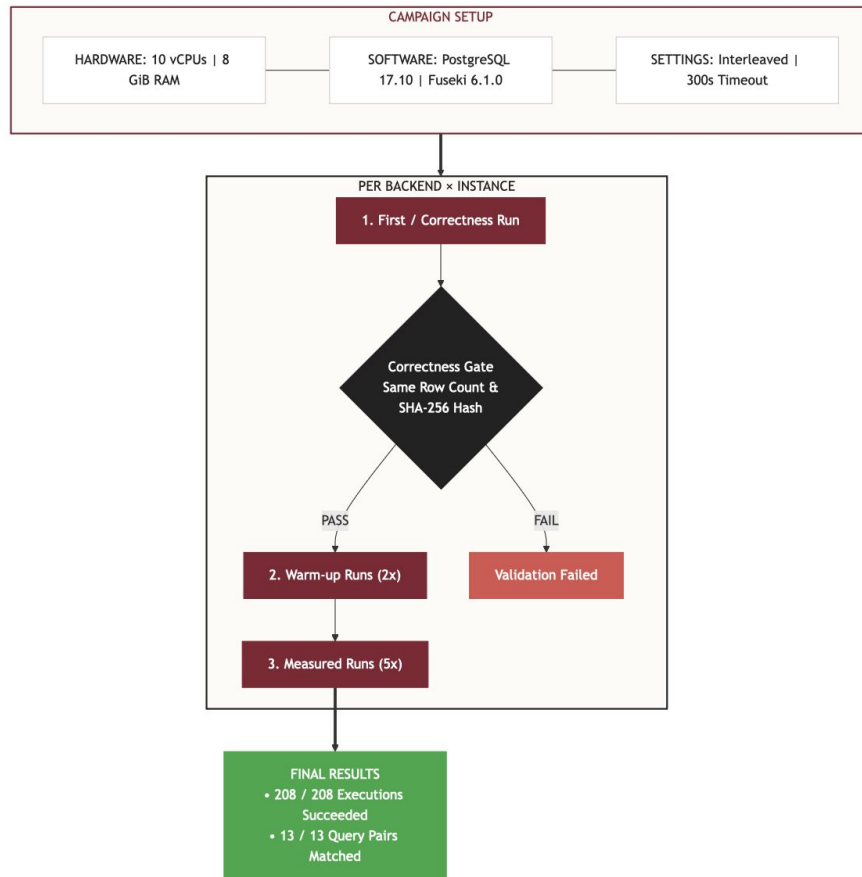


Experimental protocol

To make the evidence defensible, we had to ensure there was semantic equality between the two backends.

We propose the following flow for evaluating and comparing the Fuseki vs. PostgreSQL.

Both timings include complete client-side result retrieval. All 208 executions succeeded and all 13 paired results matched.

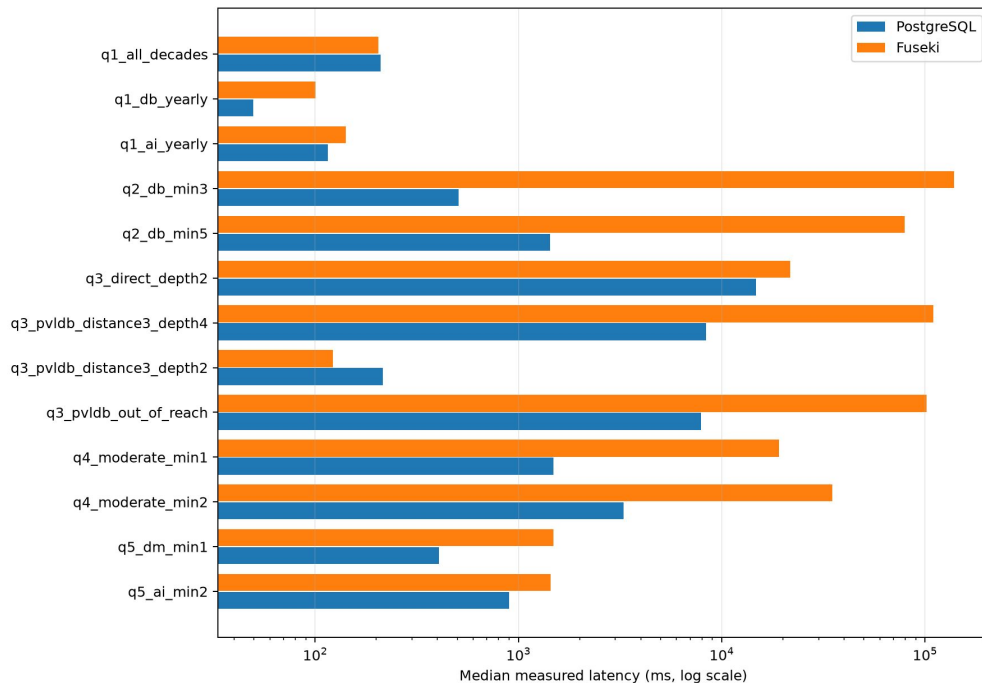


Results overview

We present the results in a logarithmic scale for readability.

PostgreSQL had the lower median in 11 of 13 instances.

- **Q1:** Close overall, all-decades is a near tie.
- **Q2:** PostgreSQL is about 56x-273x faster.
- **Q3:** Winner changes with traversal depth and graph density.
- **Q4:** PostgreSQL is about 11x-13x faster.
- **Q5:** PostgreSQL is about 1.6x-3.7x faster.



Natural Expression $\nRightarrow$ Lower Latency

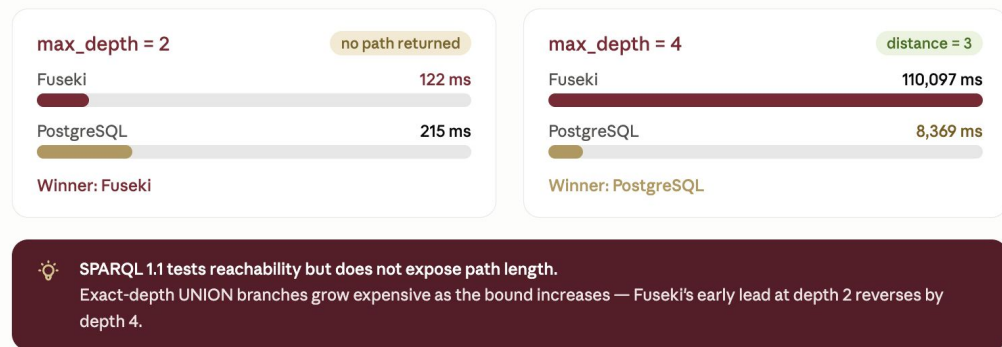
Q3 – The same query family changes winner

The two versions are not unrelated, they use the same author pair and the same PVLDB collaboration graph.

With a depth-two budget, the true distance-three target is not found and Fuseki is faster.

With a depth-four budget, the distance is returned, but PostgreSQL is about 13 times faster.

This is a clear example off why “graph traversal” is not enough for routing.



Q5 – Expressiveness and performance disagree

Q5 is our clearest expressiveness contrast.

SPARQL follows the hierarchy in one property-path expression, while SQL requires an explicit recursive closure.

PostgreSQL answers the cross-area instance faster.

We conclude that expressiveness and latency are separate axes: *the language in which a question is most natural is not necessarily the engine that executes it fastest.*

SPARQL

```
?area rdfs:subClassOf*  
  area:artificial-intelligence .
```

SQL

```
WITH RECURSIVE area_closure AS (  
  SELECT ...  
  UNION ALL  
  SELECT ... JOIN area_closure ...  
)
```

Artificial-intelligence, minimum 2 areas

winner: PostgreSQL

Fuseki

1,437 ms

PostgreSQL

902 ms



SPARQL expresses the hierarchy walk more directly with a single property-path operator, but PostgreSQL's recursive CTE executes faster here.



The advisor is an experiment, not a victory lap

We froze a transparent policy before measurement: hierarchy and traversal fo to Fuseki; grouped relational work goes to PostgreSQL.

The advisor then explains which rule fired and, when possible, reports its latency regret[†].

Takeaway: coarse structural features are insufficient, and a useful router would need empirical costs, selectivity, and cardinality

[†] Latency regret = chosen median – best observed median

Can query structure predict the faster backend?

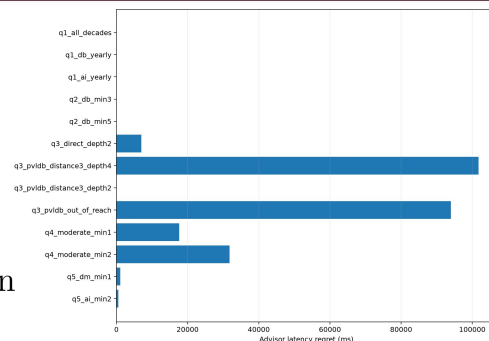
Hierarchy
→ Fuseki

Variable-length traversal
→ Fuseki

Grouped shallow/self joins
→ PostgreSQL

Policy	Correct choices	Total latency regret
Structural advisor	5 / 13	253,706 ms
Always PostgreSQL	11 / 13	98 ms
Always Fuseki	2 / 13	470,211 ms

Finding: structural intuition alone was an inadequate router.



What the evidence does and does not support

Supported

- Equivalent answers for all 13 frozen instances
- Controlled medians for this dataset and environment
- Query parameters can reverse the measured winner
- Structural routing can incur large avoidable latency

Not Supported

- A universal SQL-versus-RDF ranking
- Claims about other triplestores or configurations
- A single causal explanation for timing variability

22/26 timing series exceeded the predeclared 10% spread threshold. All runs were retained; four winners changed relative to the archived campaign.





Conclusion

*“Performance comparison
starts with semantic parity”*

- Semantic equivalence must come before performance comparison.
- Expressiveness and latency are different axes.
- Backend routing needs measured cost and data statistics, not query-shape intuition alone.

ParityDB-DBLP:

Reproducible evidence for an
explainable choice





References

- Papers:
 - [M. Schmidt et al. – "SP²Bench: A SPARQL Performance Benchmark" – ICDE 2009](#)
- Dataset:
 - <https://doi.org/10.4230/dblp.rdf.ntriples.2026-06-01>
- Slides:
 - <https://github.com/pietro-nardelli/sapienza-ppt-template>
[Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)]





Thank you for the attention!

